

claude-windows / c6028e72-ebfd-4218-89a8-c4d7dd23be96

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c6028e72-ebfd-4218-89a8-c4d7dd23be96\subagents\workflows\wf_d251ad6c-472\agent-ae345c6867261f70e.jsonl`
- SHA-256: `8265a1d573d682c084a89c6b14a4696b72bd51f65fb969ec9e6c854a0c3d5872`
- Source modified: `2026-06-08T08:40:14+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_d251ad6c-472`
- session_id: `c6028e72-ebfd-4218-89a8-c4d7dd23be96`
```

Transcript

1. user (2026-06-08T08:39:01.803Z)

Read the sports-data-collector MANUAL/LOCAL ingestion CLI path.
Files: C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py (focus on the `import-local` command, `\_parse\_local\_annotations`, and any `export` command),
C:/Users/avidu/Projects/sports-data-collector/src/cli/ingest.py.
Determine, with function signatures + line anchors:
- What local/manual annotation formats does `import-local` accept today? Exact CLI flags (`--sport`, `--video-path`, `--annotations-path`, `--license`, etc.).
- What CSV columns does `\_parse\_local\_annotations` read, require, or ignore? How does it handle JSON vs CSV?
- After parsing, how does it validate -> register (DB) -> export to the indexer eval format?
- Gaps for ingesting the rally-annotator's PARTIAL human CSV (columns `rally_number`, `start_time`, `end_time`, `ending_reason`, `sport`; partial coverage; fractional rally numbers).

2. assistant (2026-06-08T08:39:03.803Z)

I'll analyze the sports-data-collector's local ingestion path, focusing on the `import-local` command and its annotation parsing workflow.

3. user (2026-06-08T08:39:04.200Z)

```
1     import argparse
2     import sys
3     from src.core.config import load_config, get_database_path
4     from src.core.db import SportsDatabase
5     from src.parsers.badminton.coachai import CoachAIBadmintonParser
6
7     def main():
8         parser = argparse.ArgumentParser(description="Ingest external datasets into the
curation staging area")
9         parser.add_argument("--sport", required=True, choices=["badminton"], help="Sport
type")
10        parser.add_argument("--parser", required=True, choices=["coachai"], help="Parser
type to use")
11        parser.add_argument("--file-path", required=True, help="Path to raw dataset file")
12        parser.add_argument("--license", default="MIT", help="Dataset license type")
13        parser.add_argument("--video-url", default="https://youtube.com/watch?v=example",
help="URL to the video")
14        parser.add_argument("--match-name", help="Name of the match")
15        parser.add_argument("--video-id", help="Video unique identifier")
16
17        args = parser.parse_args()
18
19        # Load db path from the central config (falls back to the default).
```

```

20     # NOTE: this importer is intentionally badminton/CoachAI-only for now; other
21     # sports are populated via `curate import-local` or vendor golden sets.
22     db = SportsDatabase(get_database_path(load_config()))
23
24     if args.sport == "badminton" and args.parser == "coachai":
25         match_parser = CoachAIBadmintonParser()
26         try:
27             video_meta, rallies = match_parser.parse(
28                 args.file_path,
29                 video_id=args.video_id,
30                 video_url=args.video_url,
31                 license=args.license,
32                 match_name=args.match_name
33             )
34
35             # Insert into database (by default parsed entries are marked as staging)
36             db.insert_video(video_meta)
37             count = db.insert_badminton_rallies(rallies)
38
39             print(f"Successfully ingested match '{video_meta.match_name}' into
staging:")
40             print(f" - Video ID: {video_meta.video_id}")
41             print(f" - Total Rallies: {count}")
42             print(f" - License: {video_meta.license_type.value}")
43             print(f" - Commercial safe: {video_meta.is_commercial}")
44         except Exception as e:
45             print(f"Error parsing file: {e}")
46             sys.exit(1)
47
48     if __name__ == "__main__":
49         main()
50

```

4. user (2026-06-08T08:39:04.532Z)

```

1     import os
2     import sys
3     import time
4     import json
5     import csv
6     import argparse
7     from typing import List, Dict, Any, Tuple
8     from tabulate import tabulate
9
10    from src.core.config import load_config
11    from src.core.parsing import safe_int as _safe_int, safe_float_required as _safe_float
12    from src.core.db import SportsDatabase
13    from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
14    from src.core.schemas.badminton import BadmintonRally
15    from src.core.schemas.tennis import TennisPoint
16    from src.core.schemas.cricket import CricketDelivery
17    from src.core.schemas.pickleball import PickleballRally
18    from src.core.schemas.tabletennis import TableTennisRally
19    from src.core.validator import DatasetValidator
20    from src.core.downloader import download_best_video, probe_resolution_fps,
height_to_label
21
22
23    class SportsDataCLI:
24        def __init__(self, config_path: str = "config/config.yaml"):
25            self.config_path = config_path
26            self.config = self._load_config()
27
28            # Resolve DB path
29            db_path = self.config.get("database_path", "data/sports_metadata.db")

```

```

30         self.db = SportsDatabase(db_path)
31         self.validator = DatasetValidator(config_path)
32
33     def _load_config(self) -> Dict[str, Any]:
34         if not os.path.exists(self.config_path):
35             print(f"Warning: Configuration file not found at {self.config_path}. Using
defaults.")
36         return load_config(self.config_path)
37
38     def status(self):
39         """Displays status summary of all videos in the database."""
40         # Query total count by status via the public DB read API.
41         rows = self.db.get_status_counts() # list of (sport, status, cnt)
42
43         if not rows:
44             print("\nDatabase is currently empty. Run crawlers or import local data.\n")
45             return
46
47         data = [[sport, status, cnt] for sport, status, cnt in rows]
48         print("\n=== Dataset Ingestion Status ===")
49         print(tabulate(data, headers=["Sport", "Status", "Count"], tablefmt="grid"))
50         print()
51
52     def import_local(self, args):
53         """Manually registers a local video and annotation file into the database."""
54         sport_str = args.sport.lower()
55         if sport_str not in self.config.get("supported_sports", ["badminton"]):
56             print(f"Error: Sport '{sport_str}' is not supported. Supported:
{self.config.get('supported_sports')}")
57             sys.exit(1)
58
59         # Validate video path
60         if not os.path.exists(args.video_path):
61             print(f"Warning: Local video path '{args.video_path}' does not exist on
disk, but continuing registration.")
62
63         # Validate annotation file
64         if not os.path.exists(args.annotations_path):
65             print(f"Error: Annotations file '{args.annotations_path}' not found.")
66             sys.exit(1)
67
68         match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
69         video_id = args.video_id or match_name.lower().replace(" ", "_")
70
71         # Parse license
72         try:
73             license_type = LicenseType(args.license)
74         except ValueError:
75             print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown
(non-commercial).")
76             license_type = LicenseType.UNKNOWN
77
78         is_commercial = self.validator.check_license_commercial(license_type)
79
80         # 1. Parse Annotations
81         annotations = []
82         try:
83             annotations = self._parse_local_annotations(sport_str, video_id,
args.annotations_path)
84         except Exception as e:
85             print(f"Error parsing annotations: {e}")
86             sys.exit(1)
87
88         # 2. Run Validation
89         is_valid = True

```

```

90         errors = []
91         if sport_str == "badminton":
92             is_valid, errors = self.validator.validate_badminton_rallies(annotations)
93         elif sport_str == "tennis":
94             is_valid, errors = self.validator.validate_tennis_points(annotations)
95         elif sport_str == "cricket":
96             is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
97         elif sport_str == "pickleball":
98             is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
99         elif sport_str == "tabletennis":
100             is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
101
102         if not is_valid:
103             print(f"Error: Annotations failed validation!")
104             for err in errors:
105                 print(f" - {err}")
106             sys.exit(1)
107
108         # 3. Create Video Metadata
109         video_meta = VideoMetadata(
110             video_id=video_id,
111             sport=SportType(sport_str),
112             video_url=None,
113             local_path=args.video_path,
114             license_type=license_type,
115             license_url=None,
116             is_commercial=is_commercial,
117             status=CurationStatus.CURATED, # Custom local imports bypass staging by
default
118             match_name=match_name,
119             fps=args.fps,
120             resolution=args.resolution
121         )
122
123         # 4. Insert into Database
124         self.db.insert_video(video_meta)
125
126         count = 0
127         if sport_str == "badminton":
128             count = self.db.insert_badminton_rallies(annotations)
129         elif sport_str == "tennis":
130             count = self.db.insert_tennis_points(annotations)
131         elif sport_str == "cricket":
132             count = self.db.insert_cricket_deliveries(annotations)
133         elif sport_str == "pickleball":
134             count = self.db.insert_pickleball_rallies(annotations)
135         elif sport_str == "tabletennis":
136             count = self.db.insert_tabletennis_rallies(annotations)
137
138         print(f"\nSuccessfully imported local video '{match_name}':")
139         print(f" - Video ID: {video_id}")
140         print(f" - Sport: {sport_str}")
141         print(f" - Commercial Use Allowed: {is_commercial} (License:
{license_type.value}")
142         print(f" - Registered {count} annotated segments.\n")
143
144         # Per-video observability report (next to the video). Never raises.
145         from src.reporting import emit_import_report
146         src_fmt = os.path.splitext(args.annotations_path)[1].lstrip(".").lower() or
"unknown"
147         report_paths = emit_import_report(
148             video_meta_obj=video_meta, annotations=annotations, is_valid=True,
149             errors=[], count=count, sport=sport_str, source_format=src_fmt,
150             fallback_dir=os.path.dirname(os.path.abspath(args.annotations_path)),
151         )

```



```

216             shots_count=_safe_int(obj.get("shots_count"), None),
217             ending_reason=obj.get("ending_reason"),
218             last_shot_outcome=obj.get("last_shot_outcome")
219         ))
220     elif sport == "tabletennis":
221         parsed_items.append(TableTennisRally(
222             video_id=video_id,
223             rally_number=_safe_int(obj.get("rally_number"), idx + 1),
224             start_time=_safe_float(obj.get("start_time")),
225             end_time=_safe_float(obj.get("end_time")),
226             score_before=obj.get("score_before"),
227             score_after=obj.get("score_after"),
228             shots_count=_safe_int(obj.get("shots_count"), None),
229             ending_reason=obj.get("ending_reason"),
230             last_shot_outcome=obj.get("last_shot_outcome")
231         ))
232     elif ext == '.csv':
233         with open(filepath, 'r', encoding='utf-8') as f:
234             reader = csv.DictReader(f)
235             # Normalize headers
236             reader.fieldnames = [n.lower().strip() for n in reader.fieldnames] if
reader.fieldnames else []
237
238         for idx, row in enumerate(reader):
239             if sport == "badminton":
240                 parsed_items.append(BadmintonRally(
241                     video_id=video_id,
242                     rally_number=_safe_int(row.get("rally_number"), idx + 1),
243                     start_time=_safe_float(row.get("start_time")),
244                     end_time=_safe_float(row.get("end_time")),
245                     score_before=row.get("score_before"),
246                     score_after=row.get("score_after"),
247                     shots_count=_safe_int(row.get("shots_count"), None),
248                     ending_reason=row.get("ending_reason"),
249                     last_shot_outcome=row.get("last_shot_outcome")
250                 ))
251             elif sport == "tennis":
252                 parsed_items.append(TennisPoint(
253                     video_id=video_id,
254                     point_number=_safe_int(row.get("point_number"), idx + 1),
255                     start_time=_safe_float(row.get("start_time")),
256                     end_time=_safe_float(row.get("end_time")),
257                     set_score=row.get("set_score"),
258                     game_score=row.get("game_score"),
259                     server=row.get("server"),
260                     ending_type=row.get("ending_type")
261                 ))
262             elif sport == "cricket":
263                 parsed_items.append(CricketDelivery(
264                     video_id=video_id,
265                     over=_safe_int(row.get("over"), 0),
266                     ball=_safe_int(row.get("ball"), 0),
267                     start_time=_safe_float(row.get("start_time")),
268                     end_time=_safe_float(row.get("end_time")),
269                     batsman=row.get("batsman"),
270                     bowler=row.get("bowler"),
271                     runs=_safe_int(row.get("runs"), None),
272                     wicket=(row.get("wicket") or "").lower() in ["true", "1",
"yes"],
273                     extras=_safe_int(row.get("extras"), 0)
274                 ))
275             elif sport == "pickleball":
276                 parsed_items.append(PickleballRally(
277                     video_id=video_id,
278                     rally_number=_safe_int(row.get("rally_number"), idx + 1),

```

```

279         start_time=_safe_float(row.get("start_time")),
280         end_time=_safe_float(row.get("end_time")),
281         score_before=row.get("score_before"),
282         score_after=row.get("score_after"),
283         shots_count=_safe_int(row.get("shots_count"), None),
284         ending_reason=row.get("ending_reason"),
285         last_shot_outcome=row.get("last_shot_outcome")
286     ))
287     elif sport == "tabletennis":
288         parsed_items.append(TableTennisRally(
289             video_id=video_id,
290             rally_number=_safe_int(row.get("rally_number"), idx + 1),
291             start_time=_safe_float(row.get("start_time")),
292             end_time=_safe_float(row.get("end_time")),
293             score_before=row.get("score_before"),
294             score_after=row.get("score_after"),
295             shots_count=_safe_int(row.get("shots_count"), None),
296             ending_reason=row.get("ending_reason"),
297             last_shot_outcome=row.get("last_shot_outcome")
298         ))
299     else:
300         raise ValueError("Unsupported annotations file extension. Use .json or
.csv.")
301
302     return parsed_items
303
304 def _fetch_one_with_retry(self, url, dest_path, args, max_height, max_size_mb):
305     """Download one video at best resolution, retrying transient failures.
306
307     A below-min-height result almost always means YouTube rate-limited the
308     request and served a low fallback (not that the video lacks HD), so we
309     retry with a growing backoff. Returns (DownloadResult, None) on success,
310     or (None, failure_reason) once retries are exhausted. A below-min file is
311     discarded each attempt so it never lands in the DB.
312     """
313     attempts = max(1, args.retries + 1)
314     reason = "unknown error"
315     for i in range(attempts):
316         result = download_best_video(
317             url, dest_path,
318             max_height=max_height, max_size_mb=max_size_mb,
319             cookies_from_browser=args.cookies_from_browser,
320             cookies_file=args.cookies,
321             player_client=args.player_client,
322         )
323         if result.success and (result.height is None or result.height >=
args.min_height):
324             return result, None
325
326         if not result.success:
327             reason = result.error or "unknown error"
328         else:
329             reason = (f"got {result.resolution_label} (< min-height
{args.min_height}p) ? "
330                     f"YouTube served a low format (rate-limited?)")
331         try:
332             os.remove(result.path)
333         except OSError:
334             pass
335
336         if i < attempts - 1:
337             backoff = args.retry_backoff * (i + 1)
338             print(f" attempt {i+1}/{attempts} failed: {reason}\n retrying in
{backoff:.0f}s...")
339             time.sleep(backoff)

```

```

340         return None, reason
341
342     def fetch(self, args):
343         """(Re)download videos at best available resolution, updating the DB in place.
344
345         This is the in-place upgrade path: rows, rally annotations, and curation
346         status are all preserved ? only the video file plus its `local_path`,
347         `resolution`, and `fps` are refreshed. By default it skips videos that
348         already have a local file at or above `--min-height`, so re-runs are cheap
349         and resumable; `--force` re-fetches everything.
350         """
351         storage = self.config.get("video_storage", {})
352         local_dir = storage.get("local_dir", "./data/videos")
353         max_download_size_mb = storage.get("max_download_size_mb")
354         max_height = args.max_height if args.max_height is not None else
storage.get("max_height")
355         os.makedirs(local_dir, exist_ok=True)
356
357         commercial_only = not args.include_noncommercial
358         if args.status == "all":
359             statuses = [CurationStatus.CURATED, CurationStatus.STAGING]
360         else:
361             statuses = [CurationStatus(args.status)]
362
363         supported = self.config.get("supported_sports", ["badminton"])
364         sport_strs = [args.sport] if args.sport else supported
365
366         # Collect target videos.
367         videos = []
368         for sport_str in sport_strs:
369             try:
370                 sport = SportType(sport_str.lower())
371             except ValueError:
372                 print(f"Warning: skipping unknown sport '{sport_str}'.")
373                 continue
374             for status in statuses:
375                 videos.extend(self.db.get_videos_by_status(status, sport))
376         if args.video_id:
377             videos = [v for v in videos if v.video_id == args.video_id]
378
379         if not videos:
380             print("\nNo matching videos to fetch.\n")
381             return
382
383         results = [] # (video_id, action, detail)
384         for video in videos:
385             if commercial_only and not video.is_commercial:
386                 results.append((video.video_id, "skip", "non-commercial"))
387                 continue
388             if not video.video_url or "watch?v=example" in video.video_url:
389                 # Either no URL, or the parser's placeholder (the ShuttleSet catalog
390                 # had no URL for this match) ? nothing to download.
391                 results.append((video.video_id, "skip", "no real source URL"))
392                 continue
393
394             dest_path = os.path.join(local_dir, f"{video.video_id}.mp4")
395
396             # Skip if we already have a good-enough local file (unless --force).
397             if not args.force and os.path.exists(dest_path):
398                 _, height, _ = probe_resolution_fps(dest_path)
399                 if height is not None and height >= args.min_height:
400                     results.append((video.video_id, "have", f"{height_to_label(height)}
already"))
401                     continue
402

```

```

403         print(f"Fetching {video.video_id} ({video.match_name})...")
404         result, fail_reason = self._fetch_one_with_retry(
405             video.video_url, dest_path, args, max_height, max_download_size_mb)
406         if fail_reason is not None:
407             results.append((video.video_id, "FAIL", fail_reason))
408             if args.sleep:
409                 time.sleep(args.sleep)
410             continue
411
412         # Update only the file-derived fields; preserve everything else (UPSERT
413         # mutates the row in place, leaving rallies intact).
414         video.local_path = result.path
415         if result.resolution_label:
416             video.resolution = result.resolution_label
417         if result.fps:
418             video.fps = result.fps
419         self.db.insert_video(video)
420
421         detail = (f"{result.resolution_label or '?'}"
422                 f"{f' {result.fps:g}fps' if result.fps else ''},
423 {result.size_mb:.0f} MB")
424         results.append((video.video_id, "fetched", detail))
425         if args.sleep:
426             time.sleep(args.sleep)
427
428         print("\n=== Fetch summary ===")
429         print(tabulate([[vid, action, detail] for vid, action, detail in results],
430                        headers=["Video ID", "Action", "Detail"], tablefmt="grid"))
431         fetched = sum(1 for _, a, _ in results if a == "fetched")
432         failed = [r for r in results if r[1] == "FAIL"]
433         print(f"\nFetched/updated : {fetched}    Already-good : {sum(1 for _, a, _ in
434 results if a=='have')}}    "
435               f"Skipped : {sum(1 for _, a, _ in results if a=='skip')}}    Failed :
436 {len(failed)}}")
437         if failed:
438             print("Failures (left unchanged in DB):")
439             for vid, _, detail in failed:
440                 print(f" - {vid}: {detail}")
441         print()
442
443     def export_eval_set(self, args):
444         """Export curated annotations as indexer-ready eval/training sets.
445
446         Emits, per qualifying video, a `

```

```

465
466     for sport_str in sport_strs:
467         try:
468             sport = SportType(sport_str.lower())
469         except ValueError:
470             print(f"Warning: skipping unknown sport '{sport_str}'.")
471             continue
472
473     videos = []
474     for status in statuses:
475         videos.extend(self.db.get_videos_by_status(status, sport))
476
477     for video in videos:
478         if commercial_only and not video.is_commercial:
479             skipped_noncomm += 1
480             continue
481
482         intervals = self.db.get_segment_intervals(video.video_id, sport)
483         if not intervals:
484             skipped_empty += 1
485             continue
486
487         sport_dir = os.path.join(out_dir, sport.value)
488         os.makedirs(sport_dir, exist_ok=True)
489         csv_path = os.path.join(sport_dir, f"{video.video_id}.csv")
490         with open(csv_path, "w", newline="", encoding="utf-8") as f:
491             writer = csv.writer(f)
492             writer.writerow(["start", "end"])
493             for start, end in intervals:
494                 writer.writerow([start, end])
495
496         manifest.append({
497             "video_id": video.video_id,
498             "sport": sport.value,
499             "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
500             "local_path": video.local_path,
501             "video_url": video.video_url,
502             "match_name": video.match_name,
503             "fps": video.fps,
504             "resolution": video.resolution,
505             "license_type": video.license_type.value,
506             "is_commercial": video.is_commercial,
507             "status": video.status.value,
508             "num_segments": len(intervals),
509         })
510
511     os.makedirs(out_dir, exist_ok=True)
512     manifest_doc = {
513         "commercial_only": commercial_only,
514         "statuses": [s.value for s in statuses],
515         "total_videos": len(manifest),
516         "total_segments": sum(m["num_segments"] for m in manifest),
517         "eval_sets": manifest,
518     }
519     manifest_path = os.path.join(out_dir, "manifest.json")
520     with open(manifest_path, "w", encoding="utf-8") as f:
521         json.dump(manifest_doc, f, indent=2)
522
523     print(f"\n=== Exported eval/training set to '{out_dir}' ===")
524     if manifest:
525         summary = [[m["sport"], m["video_id"], m["num_segments"],
526                     "YES" if m["is_commercial"] else "NO",
527                     os.path.basename(m["local_path"]) if m["local_path"] else "(no
local file)"]
528                     for m in manifest]

```

```

529         print(tabulate(summary, headers=["Sport", "Video ID", "Segments", "Comm?",
"Video file"], tablefmt="grid"))
530         print(f"\nVideos exported : {len(manifest)}")
531         print(f"Total segments : {manifest_doc['total_segments']}")
532         if skipped_noncomm:
533             print(f"Skipped (non-commercial) : {skipped_noncomm} (pass --include-
noncommercial to include)")
534         if skipped_empty:
535             print(f"Skipped (no annotations) : {skipped_empty}")
536         print(f"Manifest : {manifest_path}\n")
537
538     def convert_cvat(self, args):
539         """Convert a vendor CVAT rally export into our canonical per-rally CSV.
540
541         Absorbs the CVAT bounding-box delivery format (rally fields carried as box
542         attributes) on the ingestion side: each rally's time is recomputed from
543         its frame range at the video's true fps, and content defects (overlaps,
544         zero-duration, off-vocabulary endings, coarse sampling) are written to an
545         issues report rather than silently "fixed". The emitted CSV imports via
546         `import-local`; any blocker issue must be resolved first, since the
547         importer's validator rejects overlapping / zero-duration rallies.
548         """
549         from src.parsers.cvat.rally_cvat import (
550             convert, write_canonical_csv, render_issue_report,
551         )
552         try:
553             result = convert(args.input, fps=args.fps,
554                             video_path=args.video_path, step=args.step)
555         except (FileNotFoundError, ValueError) as e:
556             print(f"Error: {e}")
557             sys.exit(1)
558
559         write_canonical_csv(result.records, args.out)
560
561         # Observability report next to the canonical CSV. Never raises.
562         from src.reporting import emit_delivery_report
563         src_name = os.path.splitext(os.path.basename(args.input))[0]
564         dur = (result.meta.get("stop_frame") / result.fps) if
(result.meta.get("stop_frame") and result.fps) else None
565         emit_delivery_report(
566             out_dir=os.path.dirname(os.path.abspath(args.out)) or ".", source=src_name,
567             records=result.records, issues=result.issues, fps=result.fps,
568             video_path=args.video_path,
569             video_duration_s=dur, source_format="cvat", step=result.step,
570         )
571         report = render_issue_report(result, os.path.basename(args.input))
572         if args.issues:
573             with open(args.issues, "w", encoding="utf-8") as f:
574                 f.write(report)
575         print(report)
576         print(f"Canonical CSV written: {args.out} ({len(result.records)} rallies)")
577         if result.n_blockers:
578             print(f"\n {result.n_blockers} BLOCKER issue(s) ? import-local will reject
"
579                 f"this file until they are resolved (by the vendor or manual
review).")
580         else:
581             print("\n No blockers. Next:")
582             print(f"    python -m src.cli.curate import-local --sport <sport> "
583                   f"--video-path <video> --annotations-path {args.out} --fps
{result.fps:g}")
584
585     def validate_delivery(self, args):
586         """Run the full validation suite on a rally delivery and emit a report.

```

```

587
588 Accepts a CVAT export (--input + --video-path/--fps) or an already-canonical
589 CSV (--csv). Writes, into --out-dir:
590     <id>_report.md    human report (scoreboard + issues + rallies to review)
591     <id>_report.json  machine summary (for CI / a bulk-ingest loop)
592     <id>_review.csv   the annotatable manual-verification sheet
593 Exits non-zero if any blocker issue exists, so it can gate a bulk pipeline.
594 """"
595 import json
596 from src.parsers.cvat.rally_cvat import (
597     convert, load_canonical_csv, detect_issues,
598     build_summary, render_validation_md, write_review_csv,
599 )
600 vid = args.video_id or "delivery"
601 fps = None
602 step = None
603 stop_frame = None
604 if args.input:
605     try:
606         result = convert(args.input, fps=args.fps, video_path=args.video_path)
607     except (FileNotFoundError, ValueError) as e:
608         print(f"Error: {e}")
609         sys.exit(1)
610     records, issues, fps = result.records, result.issues, result.fps
611     step = result.step
612     stop_frame = result.meta.get("stop_frame")
613 elif args.csv:
614     try:
615         records = load_canonical_csv(args.csv)
616     except FileNotFoundError as e:
617         print(f"Error: {e}")
618         sys.exit(1)
619     if not records:
620         print(f"Error: no usable rally rows in {args.csv}")
621         sys.exit(1)
622     issues = detect_issues(records)
623 else:
624     print("Error: provide --input <cvat.xml> or --csv <canonical.csv>")
625     sys.exit(1)
626
627 os.makedirs(args.out_dir, exist_ok=True)
628 summary = build_summary(records, issues, source=vid, fps=fps)
629 md = render_validation_md(records, issues, source=vid)
630 json_path = os.path.join(args.out_dir, f"{vid}_report.json")
631 md_path = os.path.join(args.out_dir, f"{vid}_report.md")
632 review_path = os.path.join(args.out_dir, f"{vid}_review.csv")
633 with open(json_path, "w", encoding="utf-8") as f:
634     json.dump(summary, f, indent=2)
635 with open(md_path, "w", encoding="utf-8") as f:
636     f.write(md)
637 write_review_csv(records, issues, review_path)
638
639 # Observability report (envelope JSON + technical md + customer summary),
640 # colocated with the existing artifacts. Never raises.
641 from src.reporting import emit_delivery_report
642 duration_s = (stop_frame / fps) if (stop_frame and fps) else None
643 report_paths = emit_delivery_report(
644     out_dir=args.out_dir, source=vid, records=records, issues=issues, fps=fps,
645     video_path=args.video_path, video_duration_s=duration_s,
646     source_format=("cvat" if args.input else "csv"), step=step,
647 )
648
649 print(md)
650 print(f"\nWrote:\n {md_path}\n {json_path}\n {review_path}")
651 if report_paths:

```

```

652         print(f" {report_paths.get('technical_md')}\n
{report_paths.get('customer_md')}\n {report_paths.get('json')}")
653         if summary["n_blockers"]:
654             print(f"\n GATE: BLOCKED ? {summary['n_blockers']} blocker(s) must be
resolved.")
655             sys.exit(2)
656         print("\n GATE: PASS ? review the flagged rallies, then `import-local`.")
657
658     def curate(self):
659         """Interactive loop to review and curate staging video annotations."""
660         staging_videos = self.db.get_videos_by_status(CurationStatus.STAGING)
661         if not staging_videos:
662             print("\nNo staging video annotations to review.\n")
663             return
664
665         print(f"\nFound {len(staging_videos)} matches in STAGING state for curation:")
666         table_data = []
667         for idx, video in enumerate(staging_videos):
668             table_data.append([
669                 idx + 1,
670                 video.video_id,
671                 video.match_name,
672                 video.sport.value,
673                 video.license_type.value,
674                 "YES" if video.is_commercial else "NO"
675             ])
676
677         print(tabulate(table_data, headers=["#", "Video ID", "Match Name", "Sport",
"License", "Commercial Safe?"], tablefmt="grid"))
678         print("\nEnter a number (1-{}) to review details, or 'q' to
quit:".format(len(staging_videos)))
679
680         choice = input("> ").strip()
681         if choice.lower() == 'q':
682             return
683
684         try:
685             idx = int(choice) - 1
686             if idx < 0 or idx >= len(staging_videos):
687                 print("Invalid choice.")
688                 return
689             self._curate_video_interactive(staging_videos[idx])
690         except ValueError:
691             print("Invalid input.")
692
693     def _curate_video_interactive(self, video: VideoMetadata):
694         """Displays details of a specific staging video and prompts for curation
action."""
695
696         # Load and count events
697         rallies = []
698         pts = []
699         delivs = []
700         if video.sport == SportType.BADMINTON:
701             rallies = self.db.get_badminton_rallies(video.video_id)
702         elif video.sport == SportType.PICKLEBALL:
703             rallies = self.db.get_pickleball_rallies(video.video_id)
704         elif video.sport == SportType.TABLE_TENNIS:
705             rallies = self.db.get_tabletennis_rallies(video.video_id)
706         elif video.sport == SportType.TENNIS:
707             pts = self.db.get_tennis_points(video.video_id)
708         elif video.sport == SportType.CRICKET:
709             delivs = self.db.get_cricket_deliveries(video.video_id)
710
711         while True:

```

```

712         print("\n===== REVIEWING VIDEO =====")
713         print(f"Match Name      : {video.match_name}")
714         print(f"Video ID       : {video.video_id}")
715         print(f"Sport        : {video.sport.value}")
716         print(f"Video URL    : {video.video_url or 'N/A'}")
717         print(f"Local Path   : {video.local_path or 'N/A'}")
718         print(f"License      : {video.license_type.value} ({video.license_url or 'no
URL'})")
719         print(f"Commercial OK: {video.is_commercial}")
720
721         if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
SportType.TABLE_TENNIS):
722             print(f"Total Rallies: {len(rallies)}")
723             if rallies:
724                 print("Sample rallies:")
725                 sample_data = [[r.rally_number, f"{r.start_time}s - {r.end_time}s",
r.score_after, r.shots_count] for r in rallies[:5]]
726                 print(tabulate(sample_data, headers=["Rally #", "Duration", "Score",
"Shots"], tablefmt="simple"))
727             elif video.sport == SportType.TENNIS:
728                 print(f"Total Points : {len(pts)}")
729                 if pts:
730                     print("Sample points:")
731                     sample_data = [[p.point_number, f"{p.start_time}s - {p.end_time}s",
p.game_score] for p in pts[:5]]
732                     print(tabulate(sample_data, headers=["Point #", "Duration",
"Score"], tablefmt="simple"))
733             elif video.sport == SportType.CRICKET:
734                 print(f"Total Balls  : {len(delivs)}")
735                 if delivs:
736                     print("Sample deliveries:")
737                     sample_data = [[f"{d.over}.{d.ball}", f"{d.start_time}s -
{d.end_time}s", d.runs, "Wicket" if d.wicket else ""] for d in delivs[:5]]
738                     print(tabulate(sample_data, headers=["Over.Ball", "Duration",
"Runs", "Wicket"], tablefmt="simple"))
739
740         print("\nChoose Curation Action:")
741         print("  [a] Approve (promote to CURATED)")
742         print("  [r] Reject (delete from database)")
743         print("  [p] Play a rally/point segment in browser")
744         print("  [k] Keep in Staging (do nothing and go back)")
745
746         action = input("> ").strip().lower()
747         if action == 'a':
748             self.db.update_status(video.video_id, CurationStatus.CURATED)
749             print(f"Successfully promoted '{video.match_name}' to CURATED.\n")
750             break
751         elif action == 'r':
752             self.db.delete_video(video.video_id)
753             print(f"Successfully rejected and deleted '{video.match_name}'.\n")
754             break
755         elif action == 'k':
756             print("Curation state unchanged.\n")
757             break
758         elif action == 'p':
759             self._play_segment_interactive(video, rallies, pts, delivs)
760         else:
761             print("Invalid action selected.")
762
763     def _play_segment_interactive(self, video: VideoMetadata, rallies:
List[BadmintonRally], pts: List[TennisPoint], delivs: List[CricketDelivery]):
764         import webbrowser
765
766         # Get start/end based on user input
767         start_time = None

```

```

768         end_time = None
769         label = "segment"
770
771         if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
SportType.TABLE_TENNIS):
772             if not rallies:
773                 print("No rallies to play.")
774                 return
775             idx_str = input(f"Enter Rally number to play (1-{len(rallies)}): ").strip()
776             try:
777                 rally_num = int(idx_str)
778                 rally = next((r for r in rallies if r.rally_number == rally_num), None)
779                 if rally:
780                     start_time = rally.start_time
781                     end_time = rally.end_time
782                     label = f"Rally {rally_num}"
783             except ValueError:
784                 pass
785         elif video.sport == SportType.TENNIS:
786             if not pts:
787                 print("No points to play.")
788                 return
789             idx_str = input(f"Enter Point number to play (1-{len(pts)}): ").strip()
790             try:
791                 pt_num = int(idx_str)
792                 pt = next((p for p in pts if p.point_number == pt_num), None)
793                 if pt:
794                     start_time = pt.start_time
795                     end_time = pt.end_time
796                     label = f"Point {pt_num}"
797             except ValueError:
798                 pass
799         elif video.sport == SportType.CRICKET:
800             if not delivs:
801                 print("No deliveries to play.")
802                 return
803             over_str = input("Enter Over number: ").strip()
804             ball_str = input("Enter Ball number: ").strip()
805             try:
806                 o_num = int(over_str)
807                 b_num = int(ball_str)
808                 d = next((x for x in delivs if x.over == o_num and x.ball == b_num),
None)
809             if d:
810                 start_time = d.start_time
811                 end_time = d.end_time
812                 label = f"Over {o_num} Ball {b_num}"
813             except ValueError:
814                 pass
815
816         if start_time is None or end_time is None:
817             print("Segment not found or invalid input.")
818             return
819
820         print(f"Playing {label} (start: {start_time}s, end: {end_time}s)...")
821
822         # Check video location
823         play_url = None
824         if video.local_path and os.path.exists(video.local_path):
825             abs_path = os.path.abspath(video.local_path)
826             # Create file:// URL with media fragment (browsers support #t=start,end
fragment)
827             play_url = f"file:/// {abs_path.replace(os.sep,
'/' )}#t={start_time},{end_time}"
828         elif video.video_url:

```

```

829         # YouTube URL: seek to start time
830         if "youtube.com" in video.video_url or "youtu.be" in video.video_url:
831             t_sec = int(start_time)
832             sep = "&" if "?" in video.video_url else "?"
833             play_url = f"{video.video_url}{sep}t={t_sec}s"
834         else:
835             play_url = video.video_url
836
837     if play_url:
838         print(f"Opening browser: {play_url}")
839         webbrowser.open(play_url)
840     else:
841         print("Error: No local video file or remote URL available to play.")
842
843
844     def main():
845         parser = argparse.ArgumentParser(description="Sports Data Curator CLI")
846         subparsers = parser.add_subparsers(dest="command")
847
848         # Status command
849         subparsers.add_parser("status", help="Display summary status of sports metadata
database")
850
851         # Curate command
852         subparsers.add_parser("curate", help="Launch interactive curation tool for staging
entries")
853
854         # Import-local command
855         import_parser = subparsers.add_parser("import-local", help="Manually import a local
video and annotations")
856         import_parser.add_argument("--sport", required=True, help="Sport type (e.g.
badminton, tennis, cricket)")
857         import_parser.add_argument("--video-path", required=True, help="Path to local video
file")
858         import_parser.add_argument("--annotations-path", required=True, help="Path to local
JSON/CSV annotations file")
859         import_parser.add_argument("--match-name", help="Name of the match (defaults to
video filename)")
860         import_parser.add_argument("--video-id", help="Custom alphanumeric video ID")
861         import_parser.add_argument("--license", default="Proprietary", help="License type
(e.g. MIT, CC-BY-NC, Proprietary)")
862         import_parser.add_argument("--fps", type=float, default=30.0, help="Frames per
second of the video")
863         import_parser.add_argument("--resolution", default="1080p", help="Resolution of the
video (e.g., 1080p, 720p)")
864
865         # Fetch command ? (re)download videos at best resolution, update DB in place
866         fetch_parser = subparsers.add_parser(
867             "fetch", help="(Re)download videos at best available resolution, updating the DB
in place")
868         fetch_parser.add_argument("--sport", help="Restrict to a single sport (default: all
supported_sports)")
869         fetch_parser.add_argument("--video-id", help="Fetch a single video by ID")
870         fetch_parser.add_argument("--status", choices=["staging", "curated", "all"],
default="all",
871                                     help="Which curation statuses to fetch (default: all)")
872         fetch_parser.add_argument("--min-height", type=int, default=720,
873                                     help="Skip videos already at/above this height unless
--force (default: 720)")
874         fetch_parser.add_argument("--max-height", type=int, default=None,
875                                     help="Cap download resolution (e.g. 1080); default: best
available")
876         fetch_parser.add_argument("--force", action="store_true",
877                                     help="Re-download even if a good-enough local file already
exists")

```

```

878         fetch_parser.add_argument("--include-noncommercial", action="store_true",
879                                     help="Include videos whose license is not commercially
safe (default: commercial only)")
880         fetch_parser.add_argument("--cookies-from-browser", dest="cookies_from_browser",
metavar="BROWSER",
881                                     help="Use a logged-in browser's YouTube cookies (e.g.
chrome, edge, firefox, brave). "
882                                     "A Premium login gives reliable best-resolution
downloads. "
883                                     "Chromium browsers must be CLOSED on Windows (cookie
DB lock).")
884         fetch_parser.add_argument("--cookies", metavar="FILE",
885                                     help="Path to a Netscape-format cookies.txt (alternative
to --cookies-from-browser)")
886         fetch_parser.add_argument("--player-client", dest="player_client",
default="default", metavar="CLIENT",
887                                     help="YouTube player client yt-dlp uses (default:
'default'). Avoid web/web_safari "
888                                     "(SABR-forced -> drops to 144p). 'tv'/'mweb' also
serve 1080p if needed.")
889         fetch_parser.add_argument("--sleep", type=float, default=4.0, metavar="SECONDS",
890                                     help="Pause between videos to avoid YouTube rate-limiting
(default: 4)")
891         fetch_parser.add_argument("--retries", type=int, default=2, metavar="N",
892                                     help="Retries per video when a download fails or returns a
low format (default: 2)")
893         fetch_parser.add_argument("--retry-backoff", type=float, default=20.0,
metavar="SECONDS",
894                                     help="Base backoff between retries; grows linearly per
attempt (default: 20)")
895
896         # Convert-cvat command ? normalize a vendor CVAT export into our canonical CSV
897         cvat_parser = subparsers.add_parser(
898             "convert-cvat",
899             help="Convert a vendor CVAT rally export (bounding-box XML) into the canonical
per-rally CSV")
900         cvat_parser.add_argument("--input", required=True,
901                                     help="Path to the CVAT XML export (image- or track-mode)")
902         cvat_parser.add_argument("--out", required=True,
903                                     help="Path to write the canonical per-rally CSV (import-
local input)")
904         cvat_parser.add_argument("--video-path",
905                                     help="Source video; its fps is read via ffprobe (seconds =
frame / fps)")
906         cvat_parser.add_argument("--fps", type=float,
907                                     help="Override fps instead of probing the video")
908         cvat_parser.add_argument("--step", type=int,
909                                     help="Frame sampling step (default: read from CVAT meta,
else inferred)")
910         cvat_parser.add_argument("--issues",
911                                     help="Path to also write the human-readable issues report")
912
913         # Validate-delivery command ? run the full check suite + emit a structured report
914         vd_parser = subparsers.add_parser(
915             "validate-delivery",
916             help="Validate a rally delivery (CVAT export or canonical CSV) and emit report +
review sheet")
917         vd_parser.add_argument("--input", help="CVAT XML export (use with --video-
path/--fps)")
918         vd_parser.add_argument("--csv", help="An already-canonical per-rally CSV to
validate")
919         vd_parser.add_argument("--video-path",
920                                     help="Source video; fps via ffprobe (when using --input)")
921         vd_parser.add_argument("--fps", type=float, help="Override fps (when using
--input)")

```

```

922         vd_parser.add_argument("--video-id", help="Names the report files (<id>_report.md,
etc.)")
923         vd_parser.add_argument("--out-dir", default="data/validation",
924                               help="Where to write the report + review sheet")
925
926     # Export command
927     export_parser = subparsers.add_parser(
928         "export", help="Export curated annotations as indexer-ready eval/training CSVs +
manifest")
929     export_parser.add_argument("--out-dir", default="data/eval_export",
930                               help="Output directory for per-video CSVs and
manifest.json")
931     export_parser.add_argument("--sport",
932                               help="Restrict to a single sport (default: all
supported_sports)")
933     export_parser.add_argument("--include-staging", action="store_true",
934                               help="Also export videos still in staging (default:
curated only)")
935     export_parser.add_argument("--include-noncommercial", action="store_true",
936                               help="Include videos whose license is not commercially
safe (default: commercial only)")
937
938     args = parser.parse_args()
939     cli = SportsDataCLI()
940
941     if args.command == "status":
942         cli.status()
943     elif args.command == "curate":
944         cli.curate()
945     elif args.command == "import-local":
946         cli.import_local(args)
947     elif args.command == "convert-cvat":
948         cli.convert_cvat(args)
949     elif args.command == "validate-delivery":
950         cli.validate_delivery(args)
951     elif args.command == "export":
952         cli.export_eval_set(args)
953     elif args.command == "fetch":
954         cli.fetch(args)
955     else:
956         # Default: print help
957         parser.print_help()
958
959     if __name__ == "__main__":
960         main()
961

```

5. assistant (2026-06-08T08:39:06.666Z)

Now let me look at the schema definitions to understand the required and optional fields:

6. user (2026-06-08T08:39:07.063Z)

No files found

7. user (2026-06-08T08:39:07.067Z)

No files found

8. assistant (2026-06-08T08:39:08.842Z)

Let me search more broadly:

9. user (2026-06-08T08:39:09.631Z)

/usr/bin/bash: line 1: \$null: ambiguous redirect

10. assistant (2026-06-08T08:39:11.355Z)

Let me use PowerShell instead:

11. user (2026-06-08T08:39:11.879Z)

```
C:/Users/avidu/Projects/sports-data-collector/src/core/parsing.py
C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/badminton.py
C:/Users/avidu/Projects/sports-data-collector/src/crawlers/badminton_crawler.py
C:/Users/avidu/Projects/sports-data-collector/src/parsers/badminton/coachai.py
C:/Users/avidu/Projects/sports-data-collector/tests/test_parsing.py
```

12. assistant (2026-06-08T08:39:13.584Z)

Now let me read the key files:

13. user (2026-06-08T08:39:14.096Z)

```
1      from pydantic import BaseModel, Field, model_validator
2      from typing import Optional
3
4      class BadmintonRally(BaseModel):
5          video_id: str
6          rally_number: int = Field(..., description="Chronological rally index in the match")
7          start_time: float = Field(..., description="Start of rally in seconds")
8          end_time: float = Field(..., description="End of rally in seconds")
9          score_before: Optional[str] = Field(None, description="Score before the rally (e.g.
'0-0')")
10         score_after: Optional[str] = Field(None, description="Score after the rally (e.g.
'1-0')")
11         shots_count: Optional[int] = Field(None, description="Number of strokes in the
rally")
12         ending_reason: Optional[str] = Field(None, description="Why the rally ended:
'winner' | 'forced_error' | 'unforced_error' | 'service_fault' | 'let' | 'other'")
13         last_shot_outcome: Optional[str] = Field(None, description="Where the last shot
landed: 'in' | 'net' | 'out' | 'n_a' (n_a for service_fault/let)")
14
15         @model_validator(mode='after')
16         def validate_timestamps(self) -> 'BadmintonRally':
17             if self.start_time < 0:
18                 raise ValueError("start_time must be non-negative")
19             if self.end_time <= self.start_time:
20                 raise ValueError("end_time must be strictly greater than start_time")
21             return self
22
```

14. user (2026-06-08T08:39:14.097Z)

```
1      """Shared parsing helpers used by the CLI importer and dataset parsers.
2
3      Previously these were duplicated as private ``_safe_int`` / ``_safe_float`` /
4      ``_parse_time_string`` in both ``src/cli/curate.py`` and the CoachAI parser.
5      """
6      from typing import Any, Optional
7
8
9      def safe_int(value: Any, default: Optional[int] = 0) -> Optional[int]:
10         """Parse an int from a possibly-empty/float/None value; return default on
failure."""
11         if value is None:
12             return default
13         s = str(value).strip()
```

```

14         if not s:
15             return default
16         try:
17             return int(float(s))
18         except (TypeError, ValueError):
19             return default
20
21
22     def safe_float_required(value: Any) -> float:
23         """Parse a float; raise ValueError if missing/empty (caller treats it as
24         required)."""
25         if value is None:
26             raise ValueError("missing required numeric value")
27         s = str(value).strip()
28         if not s:
29             raise ValueError("missing required numeric value")
30         return float(s)
31
32     def parse_time_string(time_str: Any) -> float:
33         """Convert 'HH:MM:SS' / 'MM:SS' / float-strings to float seconds; 0.0 if
34         blank/invalid."""
35         if not time_str:
36             return 0.0
37         s = str(time_str).strip()
38         if ":" in s:
39             parts = s.split(":")
40             try:
41                 if len(parts) == 3:
42                     return float(parts[0]) * 3600 + float(parts[1]) * 60 + float(parts[2])
43                 if len(parts) == 2:
44                     return float(parts[0]) * 60 + float(parts[1])
45             except ValueError:
46                 return 0.0
47         try:
48             return float(s)
49         except ValueError:
50             return 0.0

```

15. assistant (2026-06-08T08:39:15.938Z)

Now let me look at the validator to understand how validation works:

16. user (2026-06-08T08:39:16.631Z)

C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py

17. user (2026-06-08T08:39:18.477Z)

```

1     from typing import List, Tuple, Union
2     from src.core.config import load_config
3     from src.core.schemas.base import VideoMetadata, LicenseType
4     from src.core.schemas.badminton import BadmintonRally
5     from src.core.schemas.tennis import TennisPoint
6     from src.core.schemas.cricket import CricketDelivery
7     from src.core.schemas.pickleball import PickleballRally
8     from src.core.schemas.tabletennis import TableTennisRally
9
10    class DatasetValidator:
11        def __init__(self, config_path: str = "config/config.yaml"):
12            self.config_path = config_path
13            self.commercial_licenses = self._load_licenses_whitelist()
14

```

```

13     def _load_licenses_whitelist(self) -> List[str]:
14         # load_config returns {} on a missing/unreadable file, so the default list
15         # below is the single fallback (keep in sync with config.yaml).
16         config = load_config(self.config_path)
17         return config.get(
18             'commercial_licenses',
19             ["MIT", "Apache-2.0", "BSD", "CC0", "CC-BY", "Public-Domain",
20              "Proprietary"],
21         )
22
23     def check_license_commercial(self, license_type: Union[LicenseType, str]) -> bool:
24         """Determines if a license type is safe for commercial use based on the
25         configuration whitelist."""
26         val = license_type.value if isinstance(license_type, LicenseType) else
27         license_type
28         return val in self.commercial_licenses
29
30     def validate_badminton_rallies(self, rallies: List[BadmintonRally]) -> Tuple[bool,
31     List[str]]:
32         """
33         Validates badminton rallies:
34         - Sorted chronologically.
35         - Start time < End time (handled by Pydantic, but double checked).
36         - No temporal overlap between sequential rallies.
37         """
38         errors = []
39         if not rallies:
40             return True, errors
41
42         # Sort by rally number to check progression
43         sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
44
45         for i in range(len(sorted_rallies)):
46             r = sorted_rallies[i]
47             if r.start_time >= r.end_time:
48                 errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
49                 end_time ({r.end_time})")
50
51             if i > 0:
52                 prev = sorted_rallies[i - 1]
53                 if r.start_time < prev.end_time:
54                     errors.append(
55                         f"Rally overlap detected between Rally {prev.rally_number} "
56                         f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
57                         f"(starts at {r.start_time}s)"
58                     )
59
60         return len(errors) == 0, errors
61
62     def validate_tennis_points(self, points: List[TennisPoint]) -> Tuple[bool,
63     List[str]]:
64         errors = []
65         if not points:
66             return True, errors
67
68         sorted_pts = sorted(points, key=lambda p: p.point_number)
69         for i in range(len(sorted_pts)):
70             pt = sorted_pts[i]
71             if pt.start_time >= pt.end_time:
72                 errors.append(f"Point {pt.point_number}: start ({pt.start_time}) >= end
73                 ({pt.end_time})")
74
75             if i > 0:
76                 prev = sorted_pts[i - 1]
77                 if pt.start_time < prev.end_time:
78                     errors.append(
79                         f"Point overlap detected between Point {prev.point_number} "
80                         f"(ends at {prev.end_time}s) and Point {pt.point_number} "
81                         f"(starts at {pt.start_time}s)"
82                     )
83
84         return len(errors) == 0, errors

```

```

73             errors.append(
74                 f"Point overlap detected: Point {prev.point_number} (ends
{prev.end_time}s) "
75                 f"and Point {pt.point_number} (starts {pt.start_time}s)"
76             )
77         return len(errors) == 0, errors
78
79     def validate_cricket_deliveries(self, deliveries: List[CricketDelivery]) ->
Tuple[bool, List[str]]:
80         errors = []
81         if not deliveries:
82             return True, errors
83
84         # Sort by over, then ball within over
85         sorted_delivs = sorted(deliveries, key=lambda d: (d.over, d.ball))
86         for i in range(len(sorted_delivs)):
87             d = sorted_delivs[i]
88             if d.start_time >= d.end_time:
89                 errors.append(f"Over {d.over} Ball {d.ball}: start ({d.start_time}) >=
end ({d.end_time})")
90
91             if i > 0:
92                 prev = sorted_delivs[i - 1]
93                 if d.start_time < prev.end_time:
94                     errors.append(
95                         f"Delivery overlap: Over {prev.over} Ball {prev.ball} (ends
{prev.end_time}s) "
96                         f"and Over {d.over} Ball {d.ball} (starts {d.start_time}s)"
97                     )
98         return len(errors) == 0, errors
99
100    def validate_pickleball_rallies(self, rallies: List[PickleballRally]) -> Tuple[bool,
List[str]]:
101        """Validates pickleball rallies: positive durations and no temporal overlap
102        between chronologically-ordered rallies (same contract as badminton)."""
103        errors = []
104        if not rallies:
105            return True, errors
106
107        sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
108        for i in range(len(sorted_rallies)):
109            r = sorted_rallies[i]
110            if r.start_time >= r.end_time:
111                errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")
112
113            if i > 0:
114                prev = sorted_rallies[i - 1]
115                if r.start_time < prev.end_time:
116                    errors.append(
117                        f"Rally overlap detected between Rally {prev.rally_number} "
118                        f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
119                        f"(starts at {r.start_time}s)"
120                    )
121        return len(errors) == 0, errors
122
123    def validate_tabletennis_rallies(self, rallies: List[TableTennisRally]) ->
Tuple[bool, List[str]]:
124        """Validates table-tennis rallies: positive durations and no temporal overlap
125        between chronologically-ordered rallies (same contract as badminton)."""
126        errors = []
127        if not rallies:
128            return True, errors
129
130        sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)

```

```

131         for i in range(len(sorted_rallies)):
132             r = sorted_rallies[i]
133             if r.start_time >= r.end_time:
134                 errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")
135
136             if i > 0:
137                 prev = sorted_rallies[i - 1]
138                 if r.start_time < prev.end_time:
139                     errors.append(
140                         f"Rally overlap detected between Rally {prev.rally_number} "
141                         f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
142                         f"(starts at {r.start_time}s)"
143                     )
144             return len(errors) == 0, errors
145

```

18. assistant (2026-06-08T08:39:20.676Z)

Now let me check the export format by looking at the export_eval_set function more closely and also understand how data is stored in the database:

19. user (2026-06-08T08:39:21.327Z)

C:/Users/avidu/Projects/sports-data-collector/src/core/db.py

20. user (2026-06-08T08:39:23.426Z)

```

1     import sqlite3
2     import os
3     from contextlib import contextmanager
4     from typing import List, Dict, Any, Optional
5     from src.core.schemas.base import VideoMetadata, SportType, CurationStatus, LicenseType
6     from src.core.schemas.badminton import BadmintonRally
7     from src.core.schemas.tennis import TennisPoint
8     from src.core.schemas.cricket import CricketDelivery
9     from src.core.schemas.pickleball import PickleballRally
10    from src.core.schemas.tabletennis import TableTennisRally
11
12    # Maps each sport to its annotation table and the column(s) that order segments
13    # chronologically. All child tables expose (start_time, end_time), so a
14    # single generic reader can serve every sport ? adding a new sport is one entry
15    # here plus its CREATE TABLE, with no new query code.
16    SEGMENT_TABLES = {
17        SportType.BADMINTON: ("badminton_rallies", "rally_number"),
18        SportType.TENNIS: ("tennis_points", "point_number"),
19        SportType.CRICKET: ("cricket_deliveries", "over_num, ball_num"),
20        SportType.PICKLEBALL: ("pickleball_rallies", "rally_number"),
21        SportType.TABLE_TENNIS: ("tabletennis_rallies", "rally_number"),
22    }
23
24
25    class SportsDatabase:
26        def __init__(self, db_path: str):
27            self.db_path = db_path
28            # Ensure target directory exists
29            os.makedirs(os.path.dirname(os.path.abspath(db_path)), exist_ok=True)
30            self._init_db()
31
32        @contextmanager
33        def _get_connection(self):
34            """Yield a SQLite connection and always close it.
35
36            Used as ``with self._get_connection() as conn:``. A plain

```

```

37         ``sqlite3.connect(...)`` used as a context manager commits/rolls back the
38         transaction but does NOT close the connection, which leaks handles and
39         raises ResourceWarnings. This wrapper guarantees the connection is closed
40         on exit; writers still commit explicitly inside the block.
41         """
42         conn = sqlite3.connect(self.db_path)
43         conn.row_factory = sqlite3.Row
44         # SQLite disables foreign keys per-connection by default; enable so the
45         # ON DELETE CASCADE declarations on the child tables actually fire.
46         conn.execute("PRAGMA foreign_keys = ON")
47         try:
48             yield conn
49         finally:
50             conn.close()
51
52     def _init_db(self):
53         with self._get_connection() as conn:
54             cursor = conn.cursor()
55
56             # 1. Create common videos table
57             cursor.execute("""
58             CREATE TABLE IF NOT EXISTS videos (
59                 video_id TEXT PRIMARY KEY,
60                 sport TEXT NOT NULL,
61                 video_url TEXT,
62                 local_path TEXT,
63                 license_type TEXT NOT NULL,
64                 license_url TEXT,
65                 is_commercial INTEGER NOT NULL DEFAULT 0,
66                 status TEXT NOT NULL DEFAULT 'staging',
67                 match_name TEXT NOT NULL,
68                 fps REAL,
69                 resolution TEXT
70             )
71             """)
72
73             # 2. Create badminton_rallies table
74             cursor.execute("""
75             CREATE TABLE IF NOT EXISTS badminton_rallies (
76                 video_id TEXT NOT NULL,
77                 rally_number INTEGER NOT NULL,
78                 start_time REAL NOT NULL,
79                 end_time REAL NOT NULL,
80                 score_before TEXT,
81                 score_after TEXT,
82                 shots_count INTEGER,
83                 ending_reason TEXT,
84                 last_shot_outcome TEXT,
85                 PRIMARY KEY (video_id, rally_number),
86                 FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
87             )
88             """)
89
90             # 3. Create tennis_points table
91             cursor.execute("""
92             CREATE TABLE IF NOT EXISTS tennis_points (
93                 video_id TEXT NOT NULL,
94                 point_number INTEGER NOT NULL,
95                 start_time REAL NOT NULL,
96                 end_time REAL NOT NULL,
97                 set_score TEXT,
98                 game_score TEXT,
99                 server TEXT,
100                 ending_type TEXT,
101                 PRIMARY KEY (video_id, point_number),

```

```

102         FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
103     )
104     """)
105
106 # 4. Create cricket_deliveries table
107 cursor.execute("""
108 CREATE TABLE IF NOT EXISTS cricket_deliveries (
109     video_id TEXT NOT NULL,
110     over_num INTEGER NOT NULL,
111     ball_num INTEGER NOT NULL,
112     start_time REAL NOT NULL,
113     end_time REAL NOT NULL,
114     batsman TEXT,
115     bowler TEXT,
116     runs INTEGER,
117     wicket INTEGER DEFAULT 0,
118     extras INTEGER DEFAULT 0,
119     PRIMARY KEY (video_id, over_num, ball_num),
120     FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
121 )
122 """)
123
124 # 5. Create pickleball_rallies table
125 cursor.execute("""
126 CREATE TABLE IF NOT EXISTS pickleball_rallies (
127     video_id TEXT NOT NULL,
128     rally_number INTEGER NOT NULL,
129     start_time REAL NOT NULL,
130     end_time REAL NOT NULL,
131     score_before TEXT,
132     score_after TEXT,
133     shots_count INTEGER,
134     ending_reason TEXT,
135     last_shot_outcome TEXT,
136     PRIMARY KEY (video_id, rally_number),
137     FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
138 )
139 """)
140
141 # 6. Create tabletennis_rallies table
142 cursor.execute("""
143 CREATE TABLE IF NOT EXISTS tabletennis_rallies (
144     video_id TEXT NOT NULL,
145     rally_number INTEGER NOT NULL,
146     start_time REAL NOT NULL,
147     end_time REAL NOT NULL,
148     score_before TEXT,
149     score_after TEXT,
150     shots_count INTEGER,

```

21. user (2026-06-08T08:39:26.658Z)

```

200         video.license_url,
201         1 if video.is_commercial else 0,
202         video.status.value,
203         video.match_name,
204         video.fps,
205         video.resolution
206     ))
207     conn.commit()
208     return True
209
210 def insert_badminton_rallies(self, rallies: List[BadmintonRally]) -> int:
211     if not rallies:
212         return 0

```

```

213         with self._get_connection() as conn:
214             cursor = conn.cursor()
215             # First clear existing rallies for this video_id to avoid key conflicts on
updates
216             video_id = rallies[0].video_id
217             cursor.execute("DELETE FROM badminton_rallies WHERE video_id = ?",
(video_id,))
218
219             for rally in rallies:
220                 cursor.execute("""
221                     INSERT INTO badminton_rallies (
222                         video_id, rally_number, start_time, end_time,
223                         score_before, score_after, shots_count, ending_reason,
last_shot_outcome
224                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
225                 """, (
226                     rally.video_id,
227                     rally.rally_number,
228                     rally.start_time,
229                     rally.end_time,
230                     rally.score_before,
231                     rally.score_after,
232                     rally.shots_count,
233                     rally.ending_reason,
234                     rally.last_shot_outcome
235                 ))
236             conn.commit()
237             return len(rallies)
238
239     def insert_tennis_points(self, points: List[TennisPoint]) -> int:
240         if not points:
241             return 0
242         with self._get_connection() as conn:
243             cursor = conn.cursor()
244             video_id = points[0].video_id
245             cursor.execute("DELETE FROM tennis_points WHERE video_id = ?", (video_id,))
246
247             for pt in points:
248                 cursor.execute("""
249                     INSERT INTO tennis_points (
250                         video_id, point_number, start_time, end_time,
251                         set_score, game_score, server, ending_type
252                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
253                 """, (
254                     pt.video_id,
255                     pt.point_number,
256                     pt.start_time,
257                     pt.end_time,
258                     pt.set_score,
259                     pt.game_score,
260                     pt.server,
261                     pt.ending_type
262                 ))
263             conn.commit()
264             return len(points)
265
266     def insert_cricket_deliveries(self, deliveries: List[CricketDelivery]) -> int:
267         if not deliveries:
268             return 0
269         with self._get_connection() as conn:
270             cursor = conn.cursor()
271             video_id = deliveries[0].video_id
272             cursor.execute("DELETE FROM cricket_deliveries WHERE video_id = ?",
(video_id,))
273

```

```

274         for deliv in deliveries:
275             cursor.execute("""
276                 INSERT INTO cricket_deliveries (
277                     video_id, over_num, ball_num, start_time, end_time,
278                     batsman, bowler, runs, wicket, extras
279                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
280             """, (
281                 deliv.video_id,
282                 deliv.over,
283                 deliv.ball,
284                 deliv.start_time,
285                 deliv.end_time,
286                 deliv.batsman,
287                 deliv.bowler,
288                 deliv.runs,
289                 1 if deliv.wicket else 0,
290                 deliv.extras
291             ))
292         conn.commit()
293         return len(deliveries)
294
295     def insert_pickleball_rallies(self, rallies: List[PickleballRally]) -> int:
296         if not rallies:
297             return 0
298         with self._get_connection() as conn:
299             cursor = conn.cursor()
300             video_id = rallies[0].video_id
301             cursor.execute("DELETE FROM pickleball_rallies WHERE video_id = ?",
302 (video_id,))
303
304         for rally in rallies:
305             cursor.execute("""
306                 INSERT INTO pickleball_rallies (
307                     video_id, rally_number, start_time, end_time,
308                     score_before, score_after, shots_count, ending_reason,
309                     last_shot_outcome
310                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
311             """, (
312                 rally.video_id,
313                 rally.rally_number,
314                 rally.start_time,
315                 rally.end_time,
316                 rally.score_before,
317                 rally.score_after,
318                 rally.shots_count,
319                 rally.ending_reason,
320                 rally.last_shot_outcome
321             ))
322             conn.commit()
323             return len(rallies)
324
325     def insert_tabletennis_rallies(self, rallies: List[TableTennisRally]) -> int:
326         if not rallies:
327             return 0
328         with self._get_connection() as conn:
329             cursor = conn.cursor()
330             video_id = rallies[0].video_id
331             cursor.execute("DELETE FROM tabletennis_rallies WHERE video_id = ?",
332 (video_id,))
333
334         for rally in rallies:
335             cursor.execute("""
336                 INSERT INTO tabletennis_rallies (
337                     video_id, rally_number, start_time, end_time,
338                     score_before, score_after, shots_count, ending_reason,

```

```

last_shot_outcome
336         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
337         """ , (
338             rally.video_id,
339             rally.rally_number,
340             rally.start_time,
341             rally.end_time,
342             rally.score_before,
343             rally.score_after,
344             rally.shots_count,
345             rally.ending_reason,
346             rally.last_shot_outcome
347         ))
348     conn.commit()
349     return len(rallies)
350
351     def get_video(self, video_id: str) -> Optional[VideoMetadata]:
352         with self._get_connection() as conn:
353             cursor = conn.cursor()
354             cursor.execute("SELECT * FROM videos WHERE video_id = ?", (video_id,))
355             row = cursor.fetchone()
356             if row:
357                 return VideoMetadata(
358                     video_id=row['video_id'],
359                     sport=SportType(row['sport']),
360                     video_url=row['video_url'],
361                     local_path=row['local_path'],
362                     license_type=LicenseType(row['license_type']),
363                     license_url=row['license_url'],
364                     is_commercial=bool(row['is_commercial']),
365                     status=CurationStatus(row['status']),
366                     match_name=row['match_name'],
367                     fps=row['fps'],
368                     resolution=row['resolution']
369                 )
370             return None
371
372     def get_videos_by_status(self, status: CurationStatus, sport: Optional[SportType] =
None) -> List[VideoMetadata]:
373         query = "SELECT * FROM videos WHERE status = ?"
374         params = [status.value]
375         if sport:
376             query += " AND sport = ?"
377             params.append(sport.value)
378
379         videos = []
380         with self._get_connection() as conn:
381             cursor = conn.cursor()
382             cursor.execute(query, params)
383             for row in cursor.fetchall():
384                 videos.append(VideoMetadata(
385                     video_id=row['video_id'],
386                     sport=SportType(row['sport']),
387                     video_url=row['video_url'],
388                     local_path=row['local_path'],
389                     license_type=LicenseType(row['license_type']),
390                     license_url=row['license_url'],
391                     is_commercial=bool(row['is_commercial']),
392                     status=CurationStatus(row['status']),
393                     match_name=row['match_name'],
394                     fps=row['fps'],
395                     resolution=row['resolution']
396                 ))
397         return videos
398

```

```
399         def get_status_counts(self) -> List[tuple]:
```

22. assistant (2026-06-08T08:39:28.651Z)

Now let me check the `get_segment_intervals` method to understand the export format:

23. user (2026-06-08T08:39:29.210Z)

```
527:     def get_segment_intervals(self, video_id: str, sport: SportType) -> List[tuple]:
528-         """Return chronologically-ordered (start_time, end_time) tuples for a video,
529-         regardless of sport. This is the sport-agnostic view used to export
530-         indexer-ready evaluation/training labels.
531-
532-         Sports that have no annotation table yet (e.g. tabletennis, pickleball)
533-         yield an empty list rather than raising.
534-         """
535-         mapping = SEGMENT_TABLES.get(sport)
536-         if mapping is None:
537-             return []
538-         table, order_by = mapping # both come from the trusted map above, never user input
539-         with self._get_connection() as conn:
540-             cursor = conn.cursor()
541-             cursor.execute(
542-                 f"SELECT start_time, end_time FROM {table} "
543-                 f"WHERE video_id = ? ORDER BY {order_by}",
544-                 (video_id,),
545-             )
546-             return [(row["start_time"], row["end_time"]) for row in cursor.fetchall()]
547-
```

24. assistant (2026-06-08T08:39:31.275Z)

Perfect! Now let me check the rally-annotator project to understand what format the CSV has:

25. user (2026-06-08T08:39:31.777Z)

```
C:/Users/avidu/Projects/rally-annotator/CHANGELOG.md
C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md
C:/Users/avidu/Projects/rally-annotator/docs/ENDING_REASONS.md
C:/Users/avidu/Projects/rally-annotator/README.md
```

26. user (2026-06-08T08:39:33.831Z)

```
1         # Output CSV format
2
3         One row per rally. Header is written once when the file is created.
4
5         ```
6         rally_number,start_time,end_time,ending_reason,sport
7         1,8.800,11.500,winner,badminton
8         2,24.389,46.589,unforced_error,badminton
9         3,49.183,54.683,let,badminton
10        ```
11
12        | Column | Type | Notes |
13        |---|---|---|
14        | `rally_number` | int | 1-based, monotonic; continues across re-opening the same CSV |
15        | `start_time` | float | rally start, **decimal seconds** from the video start |
16        | `end_time` | float | rally end, decimal seconds (always `> start_time`) |
17        | `ending_reason` | enum | `winner` / `forced_error` / `unforced_error` /
18        `service_fault` / `let` / `other` ? see the decision guide in
19        [ENDING_REASONS.md](ENDING_REASONS.md) |
20        | `sport` | enum | `badminton` / `tennis` / `table_tennis` / `pickleball` / `padel` |
21
```

```

20     Notes:
21     - **Decimal seconds**, not `mm:ss` ? downstream loaders parse floats.
22     - Rows are appended live; **Undo last** truncates the last row (one level).
23     - Consumers should read by **column name** (the header is stable but column *order* may
    evolve; extra
24     columns may be added later). Most CSV `DictReader`-style loaders already tolerate
    extra columns.
25
26     ## Ingesting the labels
27
28     This CSV is intentionally compatible with common rally-segmentation tooling that reads
29     `rally_number,start_time,end_time` by header name and drops rows where `end <= start`:
30
31     ```bash
32     # Generic: any tool that takes a labels CSV of start/end seconds.
33
34     # Example ? score a detector against these labels, or slice clips by them
35     # (badminton-highlight-indexer loaders read --labels <this.csv> directly):
36     python -m backend.eval.calibrate_wasb --trajectory traj.csv --labels match.rallies.csv
    --fps 59.94 --frame-width 1920
37     python -m backend.tools.rally_slicer --video match.mp4 --labels match.rallies.csv
    --rallies all
38
39     # Example ? register as curated golden data in sports-data-collector:
40     python -m src.cli.curate import-local --sport badminton --video-path match.mp4
    --annotations-path match.rallies.csv
41     ```
42
43     The `sport` column is self-documenting; tools that take a `--sport` flag can ignore it.
44

```

27. assistant (2026-06-08T08:39:35.945Z)

Perfect! Now let me check the base schemas to understand VideoMetadata and other details:

28. user (2026-06-08T08:39:36.548Z)

C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py

29. user (2026-06-08T08:39:38.380Z)

```

1     from pydantic import BaseModel, Field
2     from typing import Optional
3     from enum import Enum
4
5     class SportType(str, Enum):
6         BADMINTON = "badminton"
7         TENNIS = "tennis"
8         TABLE_TENNIS = "tabletennis"
9         PICKLEBALL = "pickleball"
10        CRICKET = "cricket"
11
12    class LicenseType(str, Enum):
13        MIT = "MIT"
14        APACHE_2 = "Apache-2.0"
15        BSD = "BSD"
16        CC0 = "CC0"
17        CC_BY = "CC-BY"
18        CC_BY_NC = "CC-BY-NC"
19        CC_BY_NC_SA = "CC-BY-NC-SA"
20        PUBLIC_DOMAIN = "Public-Domain"
21        PROPRIETARY = "Proprietary"
22        UNKNOWN = "Unknown"
23

```

```

24     class CurationStatus(str, Enum):
25         STAGING = "staging"
26         CURATED = "curated"
27         REJECTED = "rejected"
28
29     class VideoMetadata(BaseModel):
30         video_id: str = Field(..., description="Unique alphanumeric identifier for the
video")
31         sport: SportType
32         video_url: Optional[str] = Field(None, description="Remote URL of the video (if
available)")
33         local_path: Optional[str] = Field(None, description="Local path to video file (if
downloaded or custom)")
34         license_type: LicenseType
35         license_url: Optional[str] = None
36         is_commercial: bool = Field(False, description="Flag indicating if the license
allows commercial use")
37         status: CurationStatus = CurationStatus.STAGING
38         match_name: str = Field(..., description="Name of the match/event")
39         fps: Optional[float] = Field(None, description="Frames per second")
40         resolution: Optional[str] = Field(None, description="Resolution string (e.g. 1080p,
720p)")
41

```

30. assistant (2026-06-08T08:39:41.001Z)

Excellent! Now let me create a comprehensive summary. Let me also check if there are any example CSV/JSON files to illustrate the format:

31. user (2026-06-08T08:39:41.474Z)

```

C:/Users/avidu/Projects/sports-data-
collector/data/eval_export/badminton/my_custom_local_match.csv
C:/Users/avidu/Projects/sports-data-collector/data/eval_export/badminton/shuttleaset_1.csv
C:/Users/avidu/Projects/sports-data-collector/data/eval_export/badminton/shuttleaset_10.csv
C:/Users/avidu/Projects/sports-data-collector/data/eval_export/badminton/shuttleaset_11.csv
C:/Users/avidu/Projects/sports-data-collector/data/eval_export/badminton/shuttleaset_12.csv
C:/Users/avidu/Projects/sports-data-collector/data/eval_export/badminton/shuttleaset_13.csv
C:/Users/avidu/Projects/sports-data-collector/data/eval_export/badminton/shuttleaset_14.csv
C:/Users/avidu/Projects/sports-data-collector/data/eval_export/badminton/shuttleaset_15.csv
C:/Users/avidu/Projects/sports-data-collector/data/eval_export/badminton/shuttleaset_16.csv
C:/Users/avidu/Projects/sports-data-collector/data/eval_export/badminton/shuttleaset_17.csv

```

32. user (2026-06-08T08:39:43.501Z)

```

1     start,end
2     12.5,32.0
3     45.0,61.2
4

```

33. assistant (2026-06-08T08:39:45.711Z)

Perfect! Now let me compile my comprehensive analysis with all the required information.

34. user (2026-06-08T08:40:06.748Z)

Structured output provided successfully

35. assistant (2026-06-08T08:40:14.855Z)

Summary

I've completed a thorough analysis of the sports-data-collector's local ingestion path. Here are

the key findings:

CLI Command Signature

```
```bash
python -m src.cli.curate import-local \
 --sport <badminton|tennis|cricket|pickleball|tabletennis> \
 --video-path <file.mp4> \
 --annotations-path <file.csv|file.json> \
 [--match-name "name"] \
 [--video-id custom_id] \
 [--license Proprietary] \
 [--fps 30.0] \
 [--resolution 1080p]
```
```

CSV Column Requirements

The `_parse_local_annotations()` method accepts CSV files with **case-insensitive** header matching:

- **Required:** `rally_number`` (int, or auto-indexed from row position + 1), `start_time`` (float, decimal seconds), `end_time`` (float, decimal seconds)
- **Optional:** `score_before``, `score_after``, `shots_count`` (int), `ending_reason``, `last_shot_outcome``

Missing required fields raise `ValueError``; optional fields degrade to `None``.

Validation ? Registration ? Export Pipeline

1. **Parse** (lines 156-302): CSV/JSON ? BadmintonRally objects via `safe_int/safe_float` helpers
2. **Validate** (lines 88-106): Check `start < end`, no temporal overlaps, chronological sort by `rally_number`
3. **Register** (lines 123-136): Insert `VideoMetadata` (status=`CURATED` for manual imports) + rallies into SQLite
4. **Export** (lines 440-536): Per-video CSV with `[start,end]` header + decimal-second tuples; `manifest.json` pairs CSVs with video/license metadata

Gaps for rally-annotator CSV Ingestion

The rally-annotator produces CSV with columns `(rally_number, start_time, end_time, ending_reason, sport)``:

1. **Fractional rally numbers** (e.g., 2.5 for sub-rallies): `sports-data-collector's` `safe_int()` truncates to 2, silently losing fractional precision.
2. **Partial coverage**: No validation enforcing contiguous/valid rally numbering; gaps (e.g., 1,2,4 missing 3) pass validation.
3. **Missing score fields**: rally-annotator has no `score_before`/`score_after`` ? these import as NULL, unlike full-curated sets.
4. **Sport column ignored**: rally-annotator includes sport self-documentation; `import-local` ignores it and trusts the `--sport`` flag.

The CSV format is otherwise fully compatible with the importer's expectations?ending_reason maps directly to the `BadmintonRally` schema.